

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»**

КАФЕДРА ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ



**НГТУ
НЭТИ**

**Лабораторная работа №1
по дисциплине «ТРПО-2»
на тему
«Разработка класса структуры данных на Scala»**

Группа:

Факультет: АВТФ

Студенты:

Преподаватель: Романов Е.Л.

Оглавление

1. Задание на лабораторную работу	3
---	---

Новосибирск 2022

2. Исходные данные.....	3
3. Ход работы	4
3.1. Типы данных.....	4
3.2. Паттерны/шаблоны проектирования	4
3.3.1. Фабрика (Factory).....	4
3.3.2. Объект-прототип/Строитель (Prototype/Builder)	5
3.3. Типажи (traits)	6
3.4. Структура бинарного дерева в массиве	6
3.5.1. Вставка	7
3.5.2. Поиск по индексу	8
3.5.3. Удаление	9
3.5.4. Балансировка	9
3.5.5. Итератор	11
3.5.6. Сохранение(загрузка) в(из) бинарный(ого) файл(а)	12
3.5. Пример работы разработанного ПО	12
Заключение	17
Приложение А.....	17

1. Задание на лабораторную работу

Портировать из Java разработанный в л.р.1 программный код. Сортировку реализовать с использованием техники функционального программирования и в императивном стиле. Алгоритм сортировки:

- Рекурсивное разделение;
- Рекурсивное слияние;
- Циклическое слияние;
- Однократное слияние;
- Лексикографическая сортировка;

Способы реализации операции сравнения в шаблоне:

- переопределенная операция сравнения в шаблоне;
- параметр метода сортировки – внешняя функция сравнения;

Разработать оконное приложение, работающее со структурой данных. Функции: отображение состояния (содержимого), все операции, сохранение и загрузка из текстового (двоичного) файла. Варианты реализации:

- Оконные классы в Scala – ScalaFX;
- «ручная» генерация разметки в Scala с использованием классов Java (awt или javax);
- Оконные классы в Java, передача событий из класса Scala в Java через интерфейс обратного вызова (CallBack).

2. Исходные данные

Согласно варианту задания на лабораторную работу имеем:

- Структура данных: бинарное дерево на основе массива;
- Типы данных: Целочисленный, Дата-Время.

3. Ход работы

3.1. Типы данных

Целочисленный тип данных реализован в классе *IntegerClass*. Данный класс содержит в себе единственное поле – значение типа *int* (целочисленный тип). Реализованы геттеры и сеттеры, переопределён метод *toString()*.

Тип Дата-Время более комплексный и реализован в классе *DateTimeClass*. Содержит в себе поля: день, месяц, год, час, минута, секунда. Все значения полей класса - целочисленные. Реализованы проверки на установку настоящей даты (проверка на високосный год и количество дней в месяце, принадлежность значения заданному интервалу). Реализованы геттеры и сеттеры, переопределён метод *toString()*.

Код реализации типов данных на языке программирования *Scala* представлен в Приложении А.

3.2. Паттерны/шаблоны проектирования

3.3.1. Фабрика (Factory)

В лабораторной работе №1 был реализован шаблон проектирования «Фабрика». В лабораторной работе №2 был портирован программный код данного шаблона проектирования из *Java* в *Scala*. Суть фабрики сводится к тому, чтобы производить объекты разного типа. В методе *getBuilderByName(name: String): ProtoType* фабрика получает на вход строку с названием выбранного типа данных, а возвращает объект-прототип. Также реализован метод *getTypeNameList: List[String]*, который возвращает список имен типов, на основе которых строится выпадающий список для выбора текущего типа данных в *GUI*, реализованного в качестве оконного класса на *ScalaFx* (листинг 1).

```
class FactoryType {  
  def getTypeNameList: List[String] = {
```

```

    val list = List("Integer", "DateTime")
    list}
def getBuilderByName(name: String): ProtoType = {
    name match {
        case "Integer" =>
            return new IntegerType
        case "DateTime" =>
            return new DateTimeType
    }
    null
}
}

```

Листинг 1. Реализация шаблона проектирования «Фабрика» на Scala

3.3.2. Объект-прототип/Строитель (Prototype/Builder)

В лабораторной работе №1 был реализован шаблон проектирования «Объект-прототип/Строитель». В лабораторной работе №2 был портирован программный код данного шаблона проектирования из *Java* в *Scala*. Суть его в том, чтобы обеспечить дополнительную обёртку для классов типов данных. Для реализации данного паттерна были переопределены для каждого типа данных все стандартные методы, а именно: *typeName()*, *create()*, *clone()*, *readValue()*, *parseValue()*, *getTypeComparator()*. Назначение каждого метода описано в комментариях к реализации шаблона проектирования (листинг 2). В конечном итоге были реализованы ещё два класса, наследующие типаж (trait) *ProtoType*: *IntegerType* и *DateTimeType*.

Код реализации этих классов на языке программирования *Scala* приведён в Приложении А.

```

trait ProtoType {
    // Имя типа
    def typeName: String
    // Создание объекта
    def create: AnyRef
    // Клонирование текущего
    def clone(obj: AnyRef): AnyRef
    // Создание и чтения объекта
    def readValue(inputStream: InputStream): AnyRef
    // Создает и парсит содержимое из строки
    def parseValue(someString: String): AnyRef
    // Возврат компаратора для сравнения

```

```
def getTypeComparator: Comparator  
{
```

Листинг 2. Типаж (trait) для шаблона «Объект-прототип/Строитель»

3.3. Типажи (traits)

Помимо уже рассмотренного типажа (trait) *ProtoType*, был реализован типаж компаратора *Comparator* (листинг 3). Данный типаж содержит в себе только один метод для переопределения – *compare(o1: Any, o2: Any): Int*, который позволяет сравнить значения соответствующего типа данных. Были реализованы два класса наследующий данный типаж: *IntegerComparator* (используется для сравнения целочисленных значений) и *DateTimeComparator* (используется для сравнений значений типа данных Дата-Время). Код реализации этих классов на языке программирования *Scala* приведён в Приложении А.

```
trait Comparator {  
  def compare(o1: Any, o2: Any): Int  
}
```

Листинг 2. Типаж (trait) для реализации компаратора

3.4. Структура бинарного дерева в массиве

Структура бинарного дерева была подробно рассмотрена в лабораторной работе №1.

Класс *BinaryTreeArray* – бинарного дерева в массиве содержит следующие поля:

- *var arrayTree: util.ArrayList[AnyRef]* – исходный динамический массив, который хранит объекты – узлы дерева;
- *var size: Int* – поле, обозначающее размер массива;
- *var comparator: Comparator* – компаратор, он предназначен для сравнения объектов внутри методов класса.

Для работы со структурой реализованы следующие методы на языке программирования *Scala*:

- *save(): Unit* – запись в бинарный файл;

- *load: BinaryTreeArray* – чтение из бинарного файла;
- *addValue(value: AnyRef): Unit* – добавление объекта в дерево;
- *printTree(): Unit* – печать дерева в консоль;
- *printArray(): Unit* – печать массива в консоль;
- *getDataAtIndex(searchIndex: Int): AnyRef* – получение значения по индексу;
- *removeNodeByIndex(index: Int): Unit* – удаление по индексу;
- *forEach(func: DoWith): Unit* – итератор;
- *balance: BinaryTreeArray* – балансировка дерева;
- *toString: String* – преобразование дерева в строку (необходимо для GUI).

Код реализации класса *BinaryTreeArray* на языке программирования *Scala* приведён в Приложении А.

Рассмотрим основные алгоритмы данной структуры.

3.5.1. Вставка

Включение нового значения в двоичное дерево также базируется на ветвлении: при сравнении включаемого значения со значением в текущей вершине выбирается правая или левая ветвь до тех пор, пока не встретится свободная (листинг 3). При этом, если индекс вставки будет больше, чем размер массива, то массив необходимо увеличить.

```
// Вспомогательный метод вставки значения в массив
private def insertRecursive(current: Int, obj: AnyRef): Unit = {
  if (current >= size) { // увеличение размерности при
    выходе
      size *= 2 // за пределы массива
      for (i <- size / 2 to size) { // с обнулением новой
        части
          arrayTree.add(null)
        }
      }
  if (arrayTree.get(current) == null) {
    arrayTree.set(current, obj)
    return
  }
}
```

```

    }
    if (comparator.compare(obj, arrayTree.get(current)) < 0)
insertRecursive(2 * current + 1, obj)
    else insertRecursive(2 * current + 2, obj)
}

// Вставка значения в дерево
def addValue(value: AnyRef): Unit = {
    insertRecursive(0, value)
}

```

Листинг 3. Метод вставки нового значения в дерево

3.5.2. Поиск по индексу

Поиск заданного значения в двоичном дереве использует линейно рекурсивный алгоритм. В каждой вершине производится сравнение размера левого поддерева с искомым номером. Если искомый номер меньше, чем размер левого поддерева, то рекурсивно ищется в левом поддереве. Если нет – то от искомого индекса вычитаются число вершин левого поддерева. Далее если декремент получившейся величины совпадает с нулём, то элемент найден, а если нет, то ищется индекс в правом поддереве.

В лабораторных работах №1 и №2 реализована индексация вершин по логическому номеру начиная с нуля «слева-направо» (листинг 4).

```

// Число вершин в поддереве
private def getSize(num: Int): Int = {
    if (num >= size || arrayTree.get(num) == null) return 0
    1 + getSize(2 * num + 1) + getSize(2 * num + 2)
}

private def getDataAtIndexRecursive(searchIdx: Int, help:
Int): AnyRef = {
    var searchIndex = searchIdx
    if (searchIndex >= size || searchIndex >= getSize(help))
return null
    val cntL: Int = getSize(2 * help + 1) // число вершин в
левом поддереве
    if (searchIndex < cntL) return
getDataAtIndexRecursive(searchIndex, 2 * help + 1) //
Логический номер в левом поддереве
    searchIndex -= cntL // отбросить вершины левого поддерева

    if ( {

```



```

        searchIndex -= 1; searchIndex + 1
    } == 0) return arrayTree.get(help) // Логический номер -
номер текущей вершины
    getDataAtIndexRecursive(searchIndex, 2 * help + 2) // в
правое поддерево с остатком Логического номера

}

//нумерация "слева-направо", начинается с 0, см. сprog 8.5
def getDataAtIndex(searchIndex: Int): AnyRef =
getDataAtIndexRecursive(searchIndex, 0)

```

Листинг 4. Метод поиска значения в дереве

3.5.3. Удаление

В первую очередь происходит поиск удаляемого элемента, после нахождения которого следует этап, у которого могут быть три вариации:

1. *Удаляемый узел является листовым (не имеет потомков).*

В данном случае просто удаляем лист из дерева.

2. *Удаляемый узел имеет одного потомка.*

В таком случае мы удаляем выбранный узел, и на его место ставим его потомка. После это сдвигаем поддерева переставляемого узла в массиве так, чтобы сохранились связи.

3. *Удаляемый узел имеет двух потомков.*

В данном случае, необходимо в левом поддереве найти максимальный элемент (он же будет листом) и переставить его на место удаляемого узла.

Так как реализация удаления из дерева на языке программирования *Scala* вышла достаточно объёмная, то ознакомиться с ней можно в Приложении А с подробными комментариями.

3.5.4. Балансировка

Реализована рекурсивная балансировка на языке программирования *Scala* — выполняется обход с сохранением упорядоченной последовательности массива, при котором индексы узлов собираются в *Vector*. Затем *Vector* рекурсивно делится пополам, значение из середины

включается в новое дерево. Итоговое дерево получается сбалансированным (*sprog*, раздел 8.5). Метод возвращает новый объект *BinaryTreeArray* (листинг 5).

```
//рекурсивная балансировка
private def balance(t: util.Vector[AnyRef], a: Int, b: Int,
r: util.ArrayList[AnyRef]): Unit = {
  if (a < b) {
    val m: Int = (a + b) >>> 1 // взять строку из середины
интервала
    insertRecursive(r, 0, t.get(m))
    balance(t, m + 1, b, r) // рекурсивно выполнить для
левой и

    balance(t, a, m, r) // правой частей
  }
}

//вставка для нового аррайлист при балансировке
private def insertRecursive(t: util.ArrayList[AnyRef],
current: Int, obj: AnyRef): Unit = {
  if (current >= size) {
    size *= 2
    for (i <- size / 2 to size) {
      t.add(null)
    }
  }
  if (t.get(current) == null) {
    t.set(current, obj)
    return
  }
  if (comparator.compare(obj, t.get(current)) < 0)
insertRecursive(t, 2 * current + 1, obj)
  else insertRecursive(t, 2 * current + 2, obj)
}

//главный метод балансировки
def balance: BinaryTreeArray = {
  val sz1: Int = getSize(0)
  val newArray: util.Vector[AnyRef] = new
util.Vector[AnyRef](size) //вектор индексов
  val newArrayTree: util.ArrayList[AnyRef] = new
util.ArrayList[AnyRef](size)
  for (i <- 0 until size) {
    newArrayTree.add(null)
  }
  set(newArray, 0)
  balance(newArray, 0, sz1, newArrayTree)
}
```

```

    val balanced: BinaryTreeArray = new
    BinaryTreeArray(this.size, newArrayTree, this.comparator)
    balanced
  }

  //метод для добавления индексов в вектор
  private def set(t: util.Vector[AnyRef], n: Int): Unit = {
    if (n >= size || arrayTree.get(n) == null) return
    set(t, 2 * n + 1)
    t.add(arrayTree.get(n))
    set(t, 2 * n + 2)
  }

```

Листинг 5. Балансировка дерева

3.5.5. Итератор

Под итератором *forEach* понимается метод (листинг 6), который проходит структуру данных линейно и вызывает переданную лямбду (листинг 7) для каждого хранимого элемента данных, а если лямбда срабатывает, то возвращает.

```

// итератор forEach
def forEach(func: DoWith): Unit = {
  if (arrayTree == null || size <= 0) return
  val sz: Int = getSize(0)
  val v: util.Vector[Integer] = new
  util.Vector[Integer](size)
  setHelp(v, 0)
  for (i <- 0 until sz) {
    func.doWith(arrayTree.get(v.get(i)))
  }
}

//Вспомогательный метод обхода для forEach
private def setHelp(t: util.Vector[Integer], n: Int): Unit =
{
  if (n >= size || arrayTree.get(n) == null) return
  setHelp(t, 2 * n + 1)
  t.add(n)
  setHelp(t, 2 * n + 2)
}

```

Листинг 6. Реализация итератора

```

trait DoWith {
  def doWith(obj: Any): Unit
}

```

Листинг 7. Типаж (trait) для реализации функционально тривиальных операций над элементами списка структуры данных

3.5.6. Сохранение(загрузка) в(из) бинарный(ого) файл(а)

В классе структуры данных также имеются методы сохранения дерева в двоичный файл и загрузки его из двоичного файла (листинг 8).

```
def save(): Unit = {
    try {
        val outputStream: FileOutputStream = new
FileOutputStream("saved.ser")
        val out: ObjectOutputStream = new
ObjectOutputStream(outputStream)
        out.writeObject(this)
        out.close()
        outputStream.close()
    } catch {
        case i: IOException =>
            i.printStackTrace()
    }
}

def load: BinaryTreeArray = {
    var loadedArrayTree: BinaryTreeArray = null
    try {
        val fileIn: FileInputStream = new
FileInputStream("saved.ser")
        val in: ObjectInputStream = new
ObjectInputStream(fileIn)
        loadedArrayTree =
in.readObject.asInstanceOf[BinaryTreeArray]
        in.close()
        fileIn.close()
    } catch {
        case i: IOException =>
            i.printStackTrace()
        case e: ClassNotFoundException =>
            e.printStackTrace()
    }
    loadedArrayTree
}
```

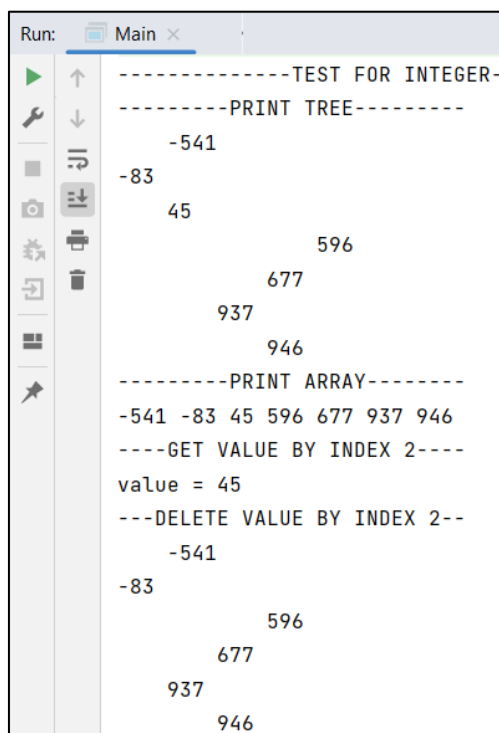
Листинг 8. Реализация сохранения в файл и загрузки из файла

3.5. Пример работы разработанного ПО

В *Main* были написаны все методы по работе с деревом для демонстрации. Результат их работы приведён на рис. 1-4.

Также в ходе лабораторной работы был реализован *GUI*, основанный на оконных классах на *ScalaFx*, по работе со структурой данных (рис. 5-6).

Код реализации класса *GUI* на *ScalaFx* приведён в Приложении А.



```
Run: Main x
-----TEST FOR INTEGER-----
-----PRINT TREE-----
-541
-83
  45
    596
    677
    937
    946
-----PRINT ARRAY-----
-541 -83 45 596 677 937 946
----GET VALUE BY INDEX 2----
value = 45
---DELETE VALUE BY INDEX 2---
-541
-83
  596
  677
  937
  946
```

Рисунок 1. Демонстрация работы операций над деревом при ТД
“IntegerClass”

```

Run: Main x
-----SAVE IN BINARY FILE-----
-----BALANCE-----
      -541
    -83
      596
    677
      937
      946
---LOAD FROM BINARY FILE---
    -541
    -83
          596
        677
      937
      946
-----FOR EACH-----
-541
-83
596
677
937
946

```

Рисунок 2. Демонстрация работы операций над деревом при ТД
“IntegerClass”

```

Run: Main x
-----TEST FOR DATETIME-----
-----PRINT TREE-----
23/1/366 06:53:37
2/9/476 00:30:39
    11/10/639 22:12:46
        8/7/804 11:39:44
            30/8/1009 05:40:54
                30/10/1017 00:34:58
                    9/10/1046 15:03:26
-----PRINT ARRAY-----
23/1/366 06:53:37 2/9/476 00:30:39 11/10/639 22:12:46 8/7/804 11:39:44 30/8/1009 05:40:54 30/10/1017 00:34:58 9/10/1046 15:03:26
----GET VALUE BY INDEX 2----
value = 11/10/639 22:12:46
---DELETE VALUE BY INDEX 2---
23/1/366 06:53:37
2/9/476 00:30:39
    8/7/804 11:39:44
        30/8/1009 05:40:54
            30/10/1017 00:34:58
                9/10/1046 15:03:26

```

Рисунок 3. Демонстрация работы операций над деревом при ТД
“DateTimeClass”

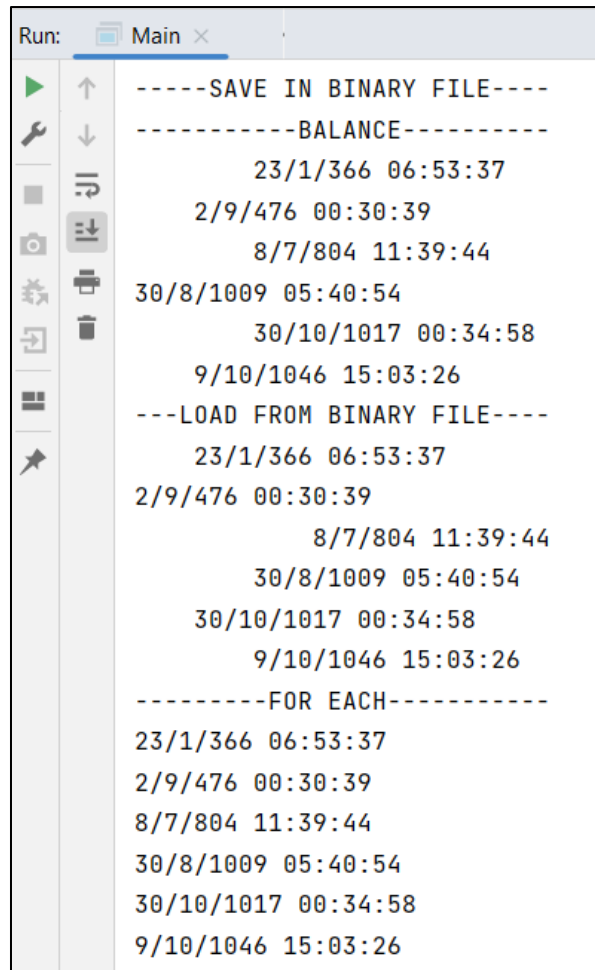


Рисунок 4. Демонстрация работы операций над деревом при ТД
“DateTimeClass”

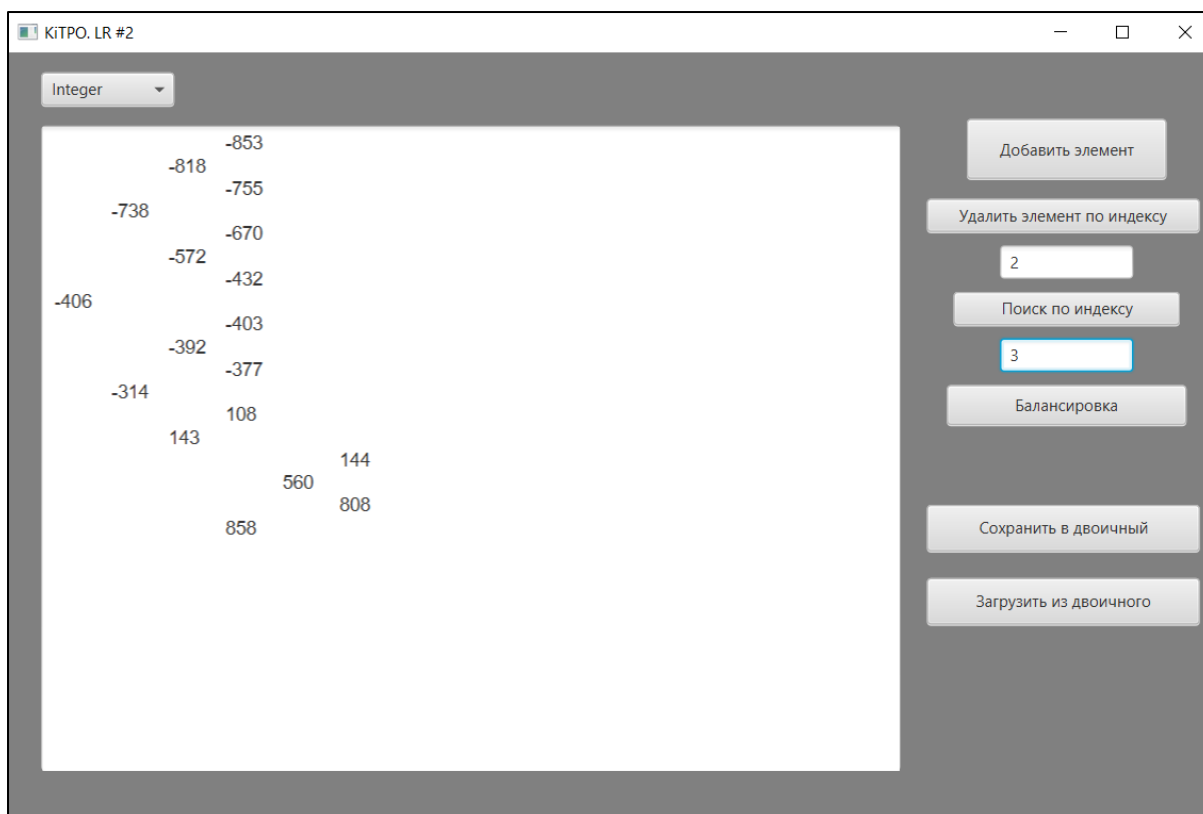


Рисунок 5. Демонстрация работы GUI, реализованного на ScalaFx

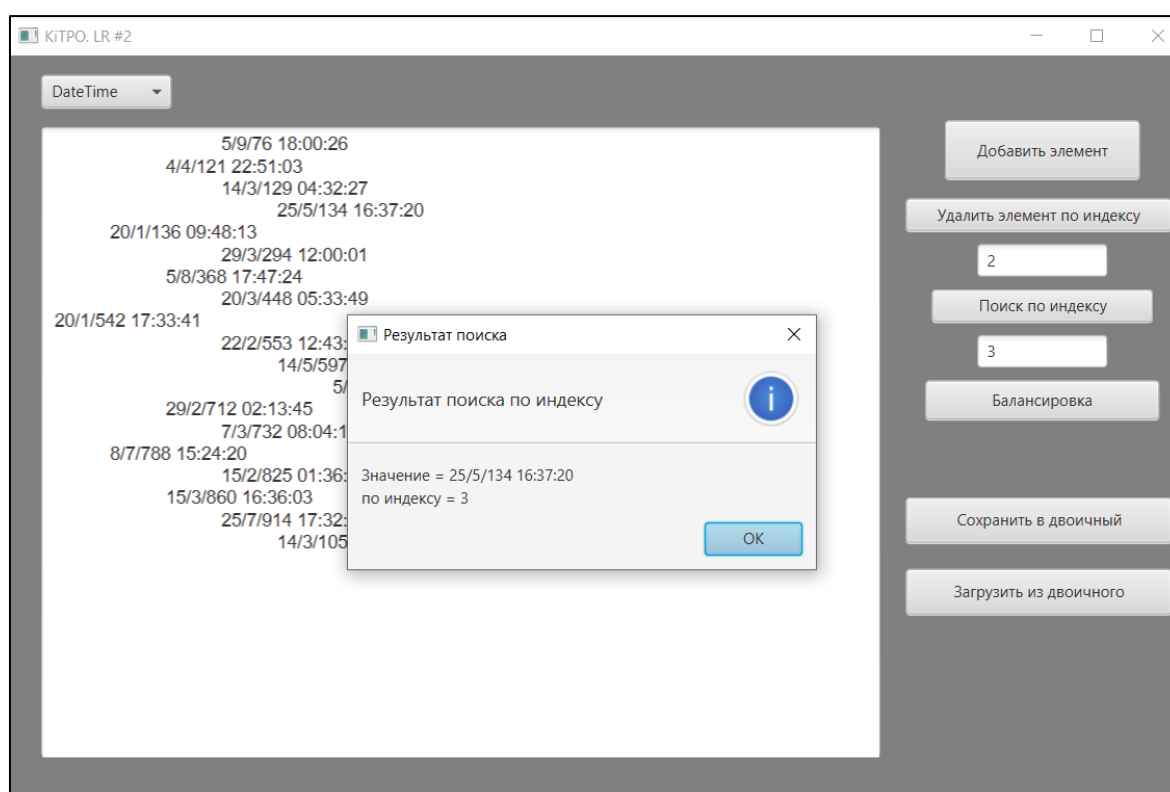


Рисунок 6. Демонстрация работы GUI, реализованного на ScalaFx

Заключение

В ходе выполнения лабораторной работы было разработано и протестировано программное обеспечение, позволяющее работать со структурой данных «Бинарное дерево в массиве» с различными типами данных (целочисленный тип и дата-время). Были применены шаблоны/паттерны проектирования «Фабрика» и «Объект-прототип/Строитель».

Программный код данного ПО был портирован с языка *Java* на *Scala*. Был сделан вывод, что возможности языка программирования *Scala* для разработки программного обеспечения являются достаточно удобными для написания кода в императивном и функциональном стилях.

Приложение А

Весь исходный код расположен по ссылке: